

OPTIMIZATION ASSIGNMENT 1

Calvin Chang 09/21/2025 **calculations done using gurobi in python*

Problem #1

Decision Variables:

Based on the question, there are 3 different variables that needs to be used when finding the maximum profit, the **amount of crude oil** purchased, the **amount of aviation fuel** sent to cracker, and the **amount of heating oil** sent to crackers, which gives the 3 variables:

$$\begin{aligned}x &\geq 0 \\a &\geq 0 \\h &\geq 0\end{aligned}\tag{1}$$

Objective function:

The objective is to maximum for Sunco's daily profit, the profit can also be known as the formula $\text{Profit} = \text{Revenue} - \text{Cost}$. Based on this formula, we can find the variables. The cost for crude oil we know is:

$$-40x\tag{2}$$

Revenue for distilled oil:

$$0.5x * 60 + 0.5x * 40 = 50x\tag{3}$$

Revenue for cracker:

$$70a + 50h \quad (4)$$

Therefore, Objective function is to **maximize** for:

$$\begin{aligned} 50x + 70a + 50h - 40x \\ = 10x + 70a + 50h \end{aligned} \quad (5)$$

Constraints:

Since all aviation fuel and heating oil must come from crude oil we know:

$$\begin{aligned} a &\leq 0.5x \\ h &\leq 0.5x \end{aligned} \quad (6)$$

And the cracker time limitations and max crude oil barrel purchases give:

$$\begin{aligned} a + 0.75h &\leq 8 \\ x &\leq 20 \end{aligned} \quad (7)$$

Optimal Solution found:

```
760.0
crude_oil 20.0
aviation_oil 8.0
heating_oil 0.0
```

Maximum profit: \$760,000

Problem #2

Decision Variables:

Based on the problem statement, we define the decision variables as follows: the number of times **process 1** is executed, the number of times **process 2** is executed, and the number of promoter hours allocated:

$$\begin{aligned}x_1, x_2 &\geq 0 \\p &\geq 0\end{aligned}\tag{8}$$

Objective function:

Objective is to **maximize** profit, which is equal to Profit = Revenue – Cost.

Revenue:

$$5(3 * x_1 + 5 * x_2)\tag{9}$$

Cost:

$$\begin{aligned}3(x_1 + 2 * x_2) + 2(2 * x_1 + 3 * x_2) + 100 * p \\= 7 * x_1 + 12 * x_2 + 100 * p\end{aligned}\tag{10}$$

Therefore, Objective function is **maximize** for:

$$\begin{aligned}15 * x_1 + 25 * x_2 - (7 * x_1 + 12 * x_2 + 100 * p) \\= 8 * x_1 + 13 * x_2 - 100 * p\end{aligned}\tag{11}$$

Constraints:

Based on constraints from max labor units, max chemical units, and demand, we can get constraint functions of:

$$\begin{aligned}
 x_1 + 2 * x_2 &\leq 20,000 \\
 2 * x_1 + 3 * x_2 &\leq 35,000 \\
 3 * x_1 + 5 * x_2 &\leq 1000 + 200 * p
 \end{aligned}
 \tag{12}$$

Optimal Solution found:

```

118000.0
process_1_runs 10000.0
process_2_runs 5000.0
promoter_hours 270.0

```

Maximum profit: \$118,000

Problem #3

Decision Variables:

Based on the question, we need the following variables:

crude oil processed by methods 1, 2, 3 correspondingly:

$$x_1, x_2, x_3 \geq 0 \tag{13}$$

oil cracked from 6 to 8 and 8 to 10:

$$c_1, c_2 \geq 0 \tag{14}$$

grade 6, 8, 10 crude oil that's blended into gas:

$$y_1, y_2, y_3 \geq 0 \tag{15}$$

grade 6, 8, 10 crude oil that's blended into heating oil:

$$z_1, z_2, z_3 \geq 0 \quad (16)$$

Objective function:

The output of the process is the total gas and the total heating oil:

$$\begin{aligned} (y_1 + y_2 + y_3) + (z_1 + z_2 + z_3) \\ G = y_1 + y_2 + y_3 \\ H = z_1 + z_2 + z_3 \end{aligned} \quad (17)$$

The revenue of the output can be found by:

$$12G + 5H \quad (18)$$

The cost of the output can be found by:

$$(3.4 * x_1 + 3.0 * x_2 + 2.6 * x_3) + (1 * c_1 + 1.5 * c_2) \quad (19)$$

Therefore, the Objective function is to **maximize** for:

$$12G + 5H - [(3.4 * x_1 + 3.0 * x_2 + 2.6 * x_3) + (1 * c_1 + 1.5 * c_2)] \quad (20)$$

Constraints:

Based on Sales capacity:

$$\begin{aligned} G &\leq 2000 \\ H &\leq 600 \end{aligned} \quad (21)$$

Average grade requirements:

$$\begin{aligned} 6 * y1 + 8 * y2 + 10 * y3 &\leq 9G \\ 6 * z1 + 8 * z2 + 10 * z3 &\leq 7H \end{aligned} \quad (22)$$

Cracking balance based on different methods:

$$\begin{aligned} y1 + z1 &= (0.3 * x1 + 0.4 * x2 + 0.1 * x3) - c1 \\ y2 + z2 &= (0.5 * x1 + 0.2 * x2 + 0.3 * x3) + c1 - c2 \\ y3 + z3 &= (0.8 * x1 + 0.4 * x2 + 0.2 * x3) + c2 \end{aligned} \quad (23)$$

Optimal Solution found:

```
21475.0
crude_oil_method1 1625.0
crude_oil_method2 0.0
crude_oil_method3 0.0
cracked6_8 0.0
cracked8_10 0.0
grade6g 187.5
grade8g 512.5
grade10g 1300.0
grade6h 300.0
grade8h 300.0
grade10h 0.0
```

Maximum profit: \$21,475

Problem #4

Decision Variables:

Based on the problem, we have variables that correspond to the amount of stock sold from stock 1 to stock 10:

$$x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10} \geq 0 \quad (24)$$

Objective function:

We simply have to **maximize** for the value of the stocks left after selling the $x_1, x_2, \dots, x_{10}$ amount of stocks:

$$\sum_{i=1}^{10} (100 - x_i) * \text{Future}_i \quad (25)$$

Constraints:

We can define c_i as the Current_i , and p_i as the Purchase_i for prices. The tax cost and transaction cost for each individual stock i sold can then be found by:

$$\begin{aligned} &0.3 * x_i (c_i - p_i) \\ &0.01 * x_i * c_i \end{aligned} \quad (26)$$

The total tax and total transaction cost is then:

$$\begin{aligned} &\sum_{i=1}^{10} 0.3 * x_i * (c_i - p_i) \\ &\sum_{i=1}^{10} (0.01 * x_i * c_i) \end{aligned} \quad (27)$$

we have to ensure the amount we sell is equal to \$300,000, Therefore the constraints are:

$$\sum_{i=1}^{10} [(x_i * c_i) - (0.3 * x_i * (c_i - p_i)) - (0.01 * x_i * c_i)] = 300,000 \quad (28)$$

Optimal Solution found:

```
20893.708807669263
sell_stock[0] 0.0
sell_stock[1] 0.0
sell_stock[2] 100.0
sell_stock[3] 100.0
sell_stock[4] 0.0
sell_stock[5] 63.75074895146793
sell_stock[6] 0.0
sell_stock[7] 100.0
sell_stock[8] 100.0
sell_stock[9] 100.0
```

Maximum future profit: \$20893.709

Problem #5

Decision Variables:

Based on the given problem, the decision variables is the amount borrowed from n credit lines for each month:

$$\begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,n} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{T,1} & x_{T,2} & \cdots & x_{T,n} \end{bmatrix} \quad (29)$$

Objective function:

The excess_{*i*} for each month *t* can then be represented as:

$$\begin{aligned} \text{payment due from 1 month ago : } R_{-1} &= r_{-1}^1 + r_{-1}^2 + \dots + r_{-1}^n \\ \text{payment due from 2 month ago : } R_{-2} &= r_{-2}^1 + r_{-2}^2 + \dots + r_{-2}^n \\ \text{payment due from 3 month ago : } R_{-3} &= r_{-3}^1 + r_{-3}^2 + \dots + r_{-3}^n \\ (1 + r) * c_{t-1} + c_t + (x_{t,1} + x_{t,2} + \dots + x_{t,n}) - R_{-1} - R_{-2} - R_{-3} \end{aligned} \quad (30)$$

Therefore, the objective function is to **maximize** the value of function:

$$(1 + r) * c_{T-1} + c_T - R_{-1} - R_{-2} - R_{-3} \quad (31)$$

Constraints:

For the given matrix of size $n * T$, every value has a lower bound ≥ 0 , and the upper bound for each line *i* cannot exceed its capacity β_i if it's still usable, or is set to zero once past the last usable month.

Collab notebook usage

I've implemented two different ways of running the function `maximize_excess_funds`, one is to enter each of the following variables manually and one is to modify the preset data dictionary to enter the values:

```
T          # number of months
n          # number of credit lines
c          # array of cash flows for T months
r          # interest rate for excess funds
beta       # Max borrow amount for each credit line
r1         # array of each r1 for each credit line
r2         # array of each r2 for each credit line
r3         # array of each r3 for each credit line
```

The function can then be called to solve the function and will output the maximum excess funds amount

AI usage

The coding, writeup and calculations for `Problem1` , `Problem2` , `Problem3` , `Problem4` were all done individually. However, while attempting `Problem 5` I was able to formulate my own functions, but had trouble formulating these functions into gurobi constraints, so I utilized GPT mainly to learn how to dynamically calculate and add constraints that were not as simple as the previous problems. Most of the code was still handwritten, the parts that used the help of **Chatgpt** are:

```
# Last usable month for each line so all repayments finish by T
last_use = []
for i in lines:
    max_lag = 0
    if r1[i] > 0: max_lag = max(max_lag, 1)
    if r2[i] > 0: max_lag = max(max_lag, 2)
    if r3[i] > 0: max_lag = max(max_lag, 3)
    last_use.append(T - max_lag if max_lag > 0 else T)

# Per-month borrowing caps
for i in lines:
    for t in months:
        borrow[i, t].UB = beta[i] if t <= last_use[i] else 0
```